SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

Table Content

1. Introduction: 4
1.1. Purpose: 4
1.2. Background: 5
1.3. Target Stakeholder 6
2. System Architecture & Design 8
2.1. High Level Architecture 8
2.1.1. Overview: 8
2.1.2 Priority-based AI Failover Service Layer 8
2.1.3 Dependency Diagram 9
2.1.4 Sequence Diagram 11
2.2. Technological Stack 14
2.3. Key Data Flows 15
2.3.1. Data Flow Diagram (DFD) 15
2.3.2. Normalized Database Schema (3NF) 16
3. Functional Requirements & Scope 17
3.1. Minimum Viable Product: 17
3.1.1. Core Features for UMHackathon 2026 Demo 17
3.2. Non-Functional Requirements (NFRs) 18
3.3. Out of Scope / Future Enhancements 19
4. Monitor, Evaluation, Assumptions & Dependencies 20
4.1. Technical Evaluation: 20
4.1.1. Grayscale Rollout & A/B Testing: 20
4.1.2. Emergency Rollback (ER) & Golden Release: 20
4.1.3. Priority Matrix: 20
4.2. Monitoring: 21
4.3. Assumptions: 21
4.4. External Dependencies: 22
5. Project Management & Team Contributions 23
5.1. Project Timeline 23
5.2. Team Members Role 25
5.3. Recommendations 25
5.3.1. Scalability 25
5.3.2. Performance 25
5.3.3. Reliability 25
5.3.4. Future Improvements 25

© UMHackathon 2026

Email: umhackathon@um.edu.my

List of Table

Table.No		Page
Table 1.1	Roles and Expectations of Target Stakeholders	6
Table 2.1	High-Level System Architecture Overview	8
Table 3.1	Core MVP Features for UMHackathon 2026 Demonstration	17
Table 3.2	Non-Functional Requirements (NFRs) and Implementation Strategies	18
Table 4.1	Operational Priority Matrix and Mitigation Strategies	20
Table 4.2	External Dependencies, Risk Assessment, and Mitigation	22
Table 5.1	Project Timeline (10-Day Sprint Plan)	23
Table 5.2	Team Members & Roles	25

© UMHackathon 2026

Email: umhackathon@um.edu.my

LIST OF ILLUSTRATIONS

Figure.No		Page
Figure 2.1	SellerMate Dual-AI System Dependency Diagram	10
Figure 2.2	Dual-AI Checkout and Structured Action Flow	12
Figure 2.3	Predictive Analytics and Graceful Degradation Pipeline	13
Figure 2.4	SellerMate Data Flow Diagram illustrating system processes and data stores (Zero Black Holes or Miracles)	15
Figure 2.5	Logical Entity Relationship Diagram (ERD) designed in strict Third Normal Form (3NF)	16

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. Introduction:

SellerMate is an AI-native social commerce operations engine designed to automate the complete fulfillment lifecycle. It serves as a strategic technical solution to the increasing operational complexity faced by sellers who manage products across multiple fragmented chat platforms. Our grand vision is to deploy this AI within a consolidated platform to centralize order data from various channels and eliminate cross-platform inventory mismanagement. For the current Minimum Viable Product (MVP) phase, a simulated Instagram interface is used to demonstrate the AI's core capabilities. The system features a Priority-based AI Failover architecture combining Z.ai and Llama 3.3 70b-verstatile (via Groq) to ensure real-time cognitive processing and predictive inventory analytics. Additionally, the system incorporates AI browser tools to fetch real-time information such as live courier rates to enable informed business decisions without human delay.

1.1. Purpose:

The primary purpose of this System Analysis Document is to detail the technical scope and architectural decisions behind the SellerMate platform. This document provides a clear technical blueprint covering the following key system elements:

1. **Architecture:**
   - A serverless monolithic infrastructure built on Next.js and Supabase, powered by a Priority-based AI Failover layer (Z.ai as primary and Groq as backup) to ensure ultra-low latency and service continuity.
2. **Data Flows:**
   - The specific mechanism detailing how unstructured conversational data from the frontend is processed by the AI engines, converted into structured intent tags, and executed as PostgreSQL database commands.
3. **Model Process:**
   - The end-to-end workflow of the AI. This includes handling customer inquiries, dynamically syncing inventory, generating daily analytical briefings, proposing optimized logistics options, and forecasting stock depletion rates.
4. **Role of Reference:**
   - A comprehensive guide for all developers to ensure strict alignment across frontend interfaces, backend API routes, and database schemas.

© UMHackathon 2026

1.2. Background:

In the modern social commerce landscape, sellers frequently distribute their products across multiple independent chat platforms and social media applications to maximize market reach.

1. **Previous Version and System Pain Points:**
   - Sellers traditionally operate in a highly manual environment, requiring them to constantly switch between different applications to answer queries and manually record orders. This fragmented approach leads to two major bottlenecks. Customers experience exceptionally long wait times for responses due to human operational delays. The lack of unified data concurrently causes disastrous inventory mismanagement, reactive restock planning, and overselling across various storefronts.

2. **Changes in Major Architectural Components:**
   - SellerMate disrupts this manual workflow by introducing a consolidated platform architecture. The system acts as a single source of truth for all inventory by utilizing a centralized Supabase database. The AI gateway intercepts the message and syncs the exact same stock data instantly regardless of the communication channel used.

3. **New capabilities:**

   The integration of the new cognitive architecture introduces unprecedented automation, delivering significant operational improvements:
   - **Priority-based AI Failover Orchestration (Z.ai + Groq)**: **Z.ai** serves as the primary engine for natural language understanding. If Z.ai experiences downtime or high latency, the system autonomously fails over to **Groq** to maintain immediate, contextually accurate responses.
   - **AI-Driven Predictive Forecasting**: The AI engine directly analyzes historical sales velocity to predict stock depletion timelines and assign dynamic risk scores. These calculations depend entirely on AI availability as there is no local mathematical fallback.
   - **Millisecond-Level Intent Recognition**: Deploys the priority-based AI pipeline to parse unstructured messages and identify customer intent with near-instant speed.
   - **Intelligent Logistics Optimization**: Utilizes AI browser tools to actively compare live courier rates across platforms to secure the lowest possible shipping costs and maximize profit margins.

© UMHackathon 2026

- **Automated Fulfillment & Tracking**: Systematically generates localized tracking messages and provides direct links to platform-specific tracking pages to ensure a transparent post-purchase experience.
- **Priority-Based Escalation**: Employs intent recognition to prioritize urgent inquiries such as payment failures or address changes, ensuring critical revenue-impacting issues are resolved before standard requests.
- **Advanced Analytical Engine**: Includes automated anomaly detection for sales spikes and generates comprehensive daily operational briefings without manual effort.
- **Human-in-the-Loop (HITL) & Bot Cut-off**: Guarantees brand safety by allowing the AI to autonomously escalate complex issues to the human owner. The system monitors the effectiveness of the priority-based failover chain (Z.ai and Groq). If this combined AI system fails to process a message for three consecutive attempts, the system autonomously cuts off the bot session and notifies the seller for immediate manual intervention.

1.3. Target Stakeholder

Table 1.1: Roles and Expectations of Target Stakeholders

Stakeholders	**Roles**	**Expectations**
Social Commerce Merchants (SMEs)	Monitor the centralized operations dashboard. They review daily analytical briefings, proactively manage restocks via predictive inventory forecasting, approve AI logistics proposals, and utilize the HITL override function (including the 3-strike bot cut-off alerts) to intervene in complex escalations.	A consolidated platform that automates routine inquiries, eliminates manual data entry, prevents overselling through predictive insights, and provides highly actionable business intelligence without requiring technical expertise.
Social Commerce Customers (End-Users)	Interact with the simulated Instagram chat interface. They inquire about product details, check availability, and confirm orders using natural language and local Malaysian slang.	Millisecond-level fast responses, highly accurate product availability, a seamless conversational checkout experience, and a guaranteed safety net for human support if

© UMHackathon 2026

		the automated system cannot resolve their issue.
Development Team	Build and maintain the serverless monolithic architecture, manage Supabase database, and orchestrate the priority-based AI Failover pipeline.	Clear API contracts, a modular codebase, and reliable failover mechanisms between Z.ai and Groq for AI stability.
QA Team	Validate the system behavior across all user roles. They focus heavily on testing the accuracy of the Dual-AI intent parsing, the reliability of the 3-strike fallback mechanism, and the integrity of database transactions during concurrent orders.	Well-defined testing scopes, strict API latency thresholds like returning AI responses within acceptable timeframes, validation of the predictive mathematical formulas, and comprehensive coverage of conversational edge cases.

© UMHackathon 2026

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Table 2.1: High-Level System Architecture Overview

Type	Details
System	Next.js 15 Web Application (Simulated Instagram Interface & Merchant Dashboard)
Architecture	Serverless Monolithic with Priority-based AI Failover Integration

SellerMate utilizes a client-server architecture where the Next.js frontend delivers two distinct interfaces: a simulated Instagram mobile chat for customers and a comprehensive management dashboard for vendors. All components communicate via a unified Backend API layer built with Next.js App Router. The system leverages a centralized Supabase PostgreSQL database as the single source of truth for inventory, conversation history, and order fulfillment data.

2.1.2 Priority-based AI Failover Service Layer

The AI architecture is integrated as the core cognitive engine, functioning as the central operations hub rather than a simple chatbot module. Its architectural responsibilities are defined as follows:

- **Intent Parsing and NLU:** The system designates **Z.ai as the primary engine** for natural language understanding. If Z.ai experiences downtime or high latency, the system autonomously fails over to the high-speed Groq inference engine to process unstructured messages, including localized Malaysian slang. This ensures ultra-low latency when distinguishing between routine inquiries and transactional commands.
- **Dynamic Context Processing:** Synthesizes dynamic system prompts injected by the backend. It combines real-time PostgreSQL database snapshots of inventory status with active conversation history to maintain stateful memory.
- **AI-Driven Predictive Inventory Forecasting:** The AI engine directly analyzes historical sales data to output dynamic risk scores and predict exact stock depletion timelines. Unlike traditional systems, this model relies entirely on AI processing to transition operations from reactive to proactive, with no local mathematical fallback.

© UMHackathon 2026

- **Advanced Analytics and Safe Failure:** Serves as an analytical engine for daily briefings and anomaly detection. The engine utilizes Regex extraction to isolate structured JSON data from AI output. In the event of an API timeout or service failure across the AI providers, the system triggers a strict 40-second circuit breaker. Instead of crashing, it safely returns a "Prediction Unavailable" state to ensure the rest of the merchant dashboard remains fully operational.
- **Structured Action Generation:** Engineered to output actionable tags such as [DEDUCT_STOCK]. The backend specifically intercepts these tokens to trigger atomic database transactions, effectively bridging natural language with rigid social commerce logic.
- **Human-in-the-Loop (HITL) State Management & Bot Cut-off:** Governs a shared conversation space via strict database states. Crucially, it incorporates a three-strike fallback mechanism: if the combined failover chain (Z.ai and Groq) fails to successfully resolve a query three consecutive times, the system automatically terminates the bot session and pushes an immediate notification to the merchant. This allows for seamless manual intervention via predefined handover protocols, guaranteeing absolute brand safety.
- **Logistics Optimization and Fulfillment Automation:** Evaluates destination regions to propose cost-effective external couriers via AI browser tools. Following merchant approval, the system autonomously generates localized tracking messages and clickable platform-specific links like Shopee to ensure a seamless post-purchase journey.

2.1.3 Dependency Diagram

The system dependency diagram below illustrates the comprehensive architecture and critical data flows of the SellerMate platform. It maps the interactions between the Next.js frontend, the centralized backend gateway, the Supabase database, and the external services layer. Specifically, the diagram highlights three highly optimized operational pillars:

- **Priority-based AI Failover Flow:** Demonstrates how the system orchestrates **Z.ai (Primary)** and **Groq (Backup)**. If the primary engine experiences latency or downtime, the system automatically re-routes the request to ensure a seamless conversational experience.
- **The Resilience & HITL Defense Flow:** Showcases the platform's defensive programming, featuring an 40-second circuit breaker and a "3-Strike" monitoring mechanism. If the AI fails to resolve a query consecutively, the system forcefully terminates the bot session and routes an emergency alert to the merchant dashboard.

© UMHackathon 2026

- **AI-Driven Analytics Flow:** Details the B2B dashboard logic where the AI engine directly analyzes raw inventory and sales data from Supabase to provide dynamic insights and proactive supply chain alerts without relying on local mathematical fallbacks.

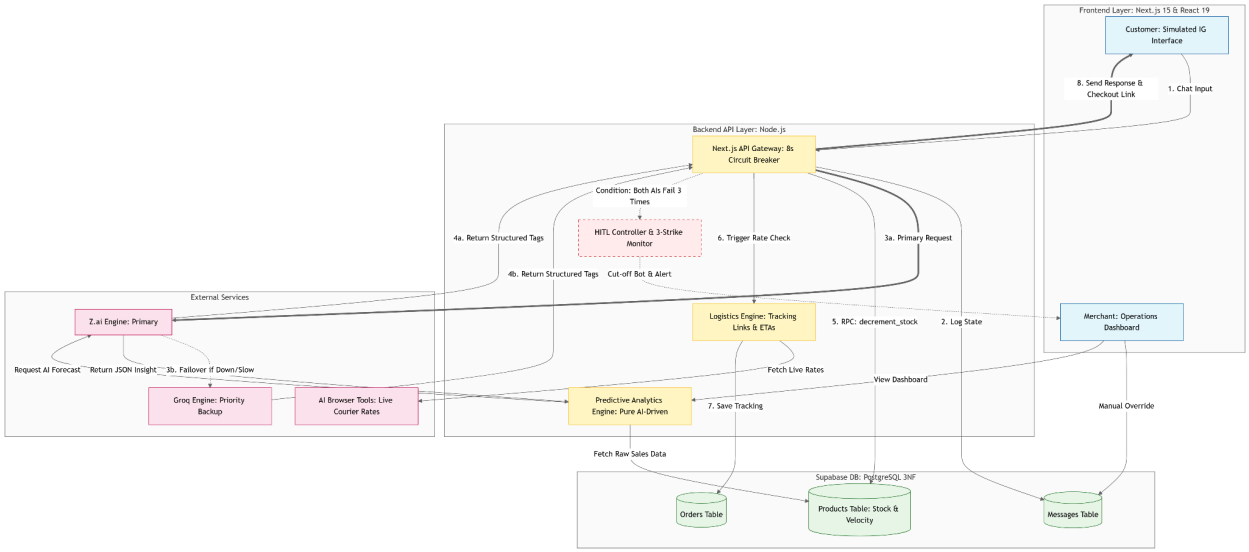

Figure 2.1: SellerMate Dual-AI System Dependency Diagram

© UMHackathon 2026

Email: umhackathon@um.edu.my

2.1.4 Sequence Diagram

a. The Ideal Path: Priority-based AI Checkout & HITL Escalation Flow

The sequence diagram below maps the ideal customer journey, seamlessly transforming unstructured Malaysian slang into rigid social commerce execution through three core phases:

- **Context Injection & Priority AI Parsing:** The API Gateway injects Supabase chat history and inventory data into **Z.ai (Primary)**. A 40-second timeout or downtime automatically triggers a failover to **Groq (Backup)** to ensure uninterrupted parsing.
- **Structured Action Execution:** The active AI outputs a definitive action tag ([DEDUCT_STOCK]). The backend immediately triggers an atomic database transaction to lock inventory and prevent overselling.
- **Intelligent Logistics:** AI Browser Tools fetch and compare live courier rates based on regional destination. The gateway then generates and dispatches the final checkout link with optimized shipping costs.

© UMHackathon 2026

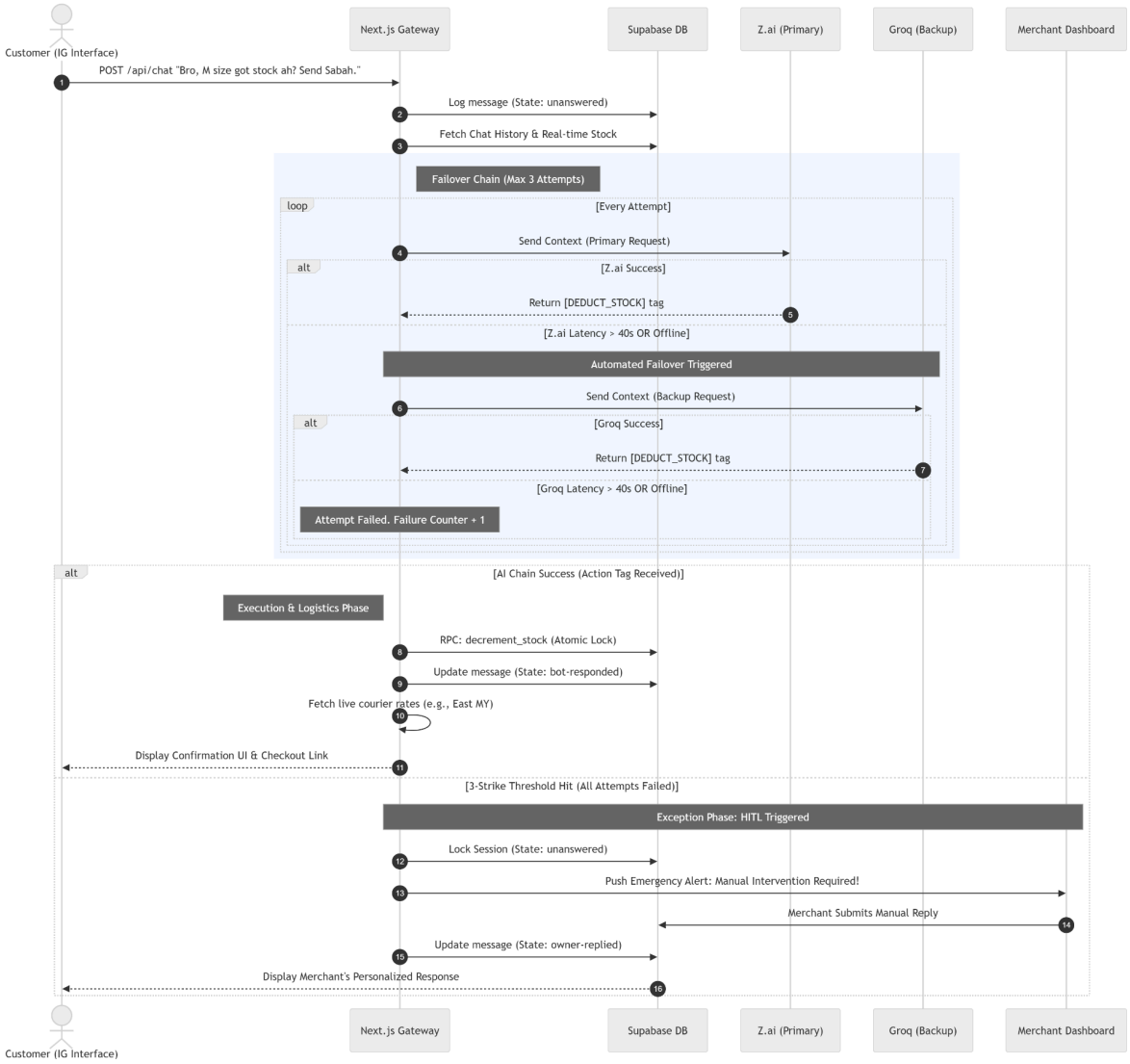

Figure 2.2: Unified AI Execution and Failover Sequence

© UMHackathon 2026

Email: umhackathon@um.edu.my

b. The Analytical Path: AI-Driven Predictive Analytics & State Management

The final sequence diagram illustrates the B2B dashboard logic for inventory forecasting. It highlights a purely AI-dependent architectural pattern:

- **Pure AI-Driven Forecasting:** The backend extracts raw historical sales data directly from Supabase and injects the payload into **Z.ai**. The cognitive engine autonomously calculates sales velocity and stock depletion timelines to generate dynamic business insights without relying on backend mathematical pre-calculations.
- **Strict Timeout & Safe Failure:** The analytical request is governed by a strict 40-second circuit breaker. Because the forecasting relies entirely on AI inference with no local mathematical fallback, a timeout or provider failure gracefully returns a "Prediction Unavailable" state. This prevents backend crashes and ensures the core dashboard remains fully accessible to the merchant.

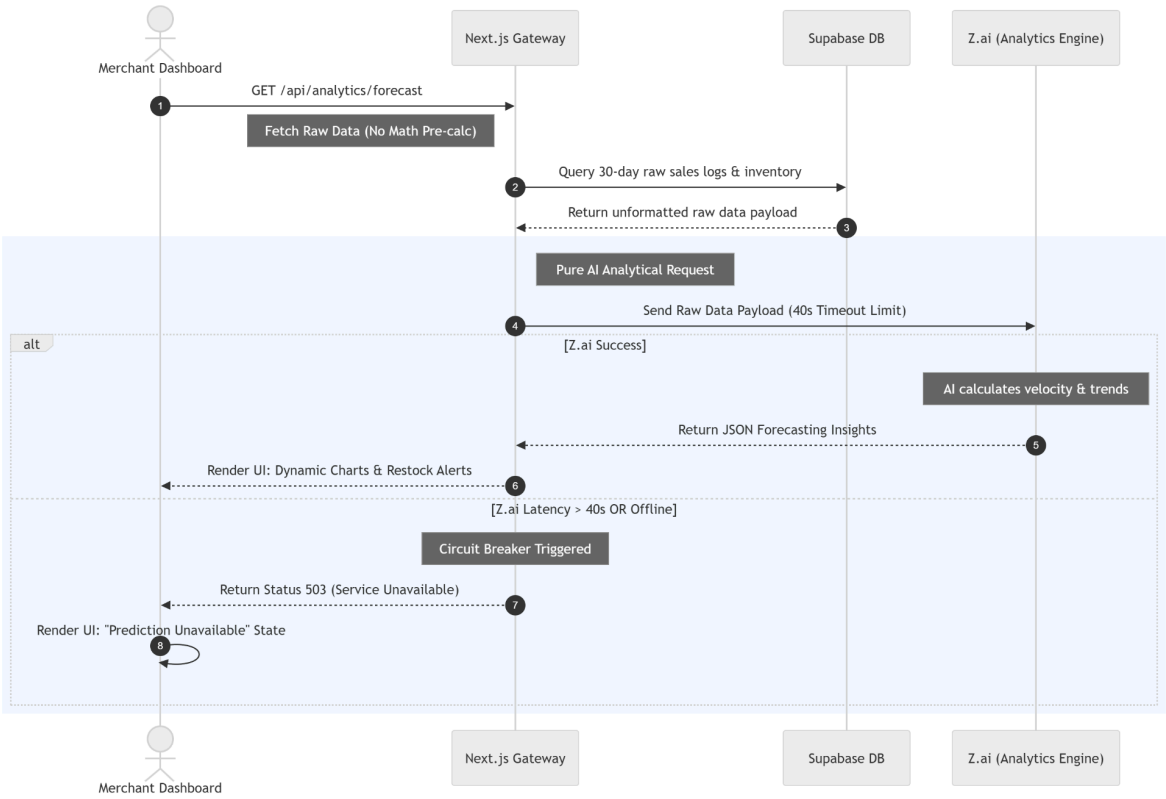

Figure 2.3: AI-Driven Analytics and Safe Failure Flow

2.2. Technological Stack

1. **Frontend:** Next.js 15 & React 19
   - Details: Utilized to build both the simulated Instagram mobile interface for customers and the interactive Operations Dashboard for merchants.
   - Justification: Next.js React Server Components (RSC) drastically reduce client-side JavaScript bundles. This ensures a highly responsive, native-like chat experience on mobile devices while maintaining SEO and performance for the B2B dashboard.
2. **Backend:** Next.js API Routes (Serverless Environment)
   - Details: Functions as the centralized API Gateway and orchestration layer.
   - Justification: A serverless approach eliminates server maintenance overhead and scales automatically with surge traffic. This layer handles the complex orchestration of the 40-second circuit breaker, the 3-strike HITL monitoring, and the hybrid reasoning pre-calculations (Math) before forwarding payloads to the AI.
3. **Cognitive Service Layer:** Priority-based AI Failover (Z.ai + Groq)
   - Details: The intelligence core of the platform, configured in a high-availability primary/backup setup to replace rigid chatbot frameworks.
   - Justification: Z.ai serves as the primary engine for natural language understanding and pure AI-driven predictive analytics. Groq (leveraging its LPU for blazing-fast token generation) acts strictly as a priority backup. If Z.ai experiences latency or downtime, the system autonomously fails over to Groq. This ensures uninterrupted parsing of unstructured Malaysian slang and consistent generation of structured execution tags.
4. **Database:** Supabase (PostgreSQL)
   - Details: Serves as the absolute single source of truth for all social commerce data, including Messages, Products, and Orders.
   - Justification: Selected for PostgreSQL's strict ACID compliance. This allows the backend to execute safe, atomic Remote Procedure Calls (RPCs)—such as decrement_stock and increment_stock—which are absolutely critical to preventing concurrency issues and overselling during checkout spikes.
5. **Cloud / Deployment:** Vercel & Supabase Cloud
   - Details: Fully managed edge computing and database hosting.
   - Justification: Vercel deploys the Next.js application to the Edge Network, ensuring millisecond-level API routing between the simulated IG interface and the AI services. Supabase Cloud provides a scalable, managed PostgreSQL instance, allowing the team to focus entirely on application logic rather than complex AWS EC2/RDS DevOps configurations.

© UMHackathon 2026

2.3. Key Data Flows

This section outlines how data systematically moves through the SellerMate architecture and how it is persistently stored. The design prioritizes strict data integrity, preventing asynchronous race conditions during high-volume checkout events.

2.3.1. Data Flow Diagram (DFD)

The Data Flow Diagram below illustrates the continuous movement of data between external entities (Customers, Merchants, External AI Services), transformation processes, and internal data stores (Supabase).

Crucially, this DFD adheres strictly to standard system analysis laws to guarantee operational validity. It is carefully designed to avoid logical errors such as "Black Holes" (processes that have inputs but produce no output) and "Miracles" (processes that generate outputs without sufficient inputs). For instance, Process 1.0 (Process Chat Intent) does not consume the customer's message into a void; it transforms the input via the Dual-AI external service and outputs both a state log to the D1: Messages data store and an actionable trigger to Process 2.0.

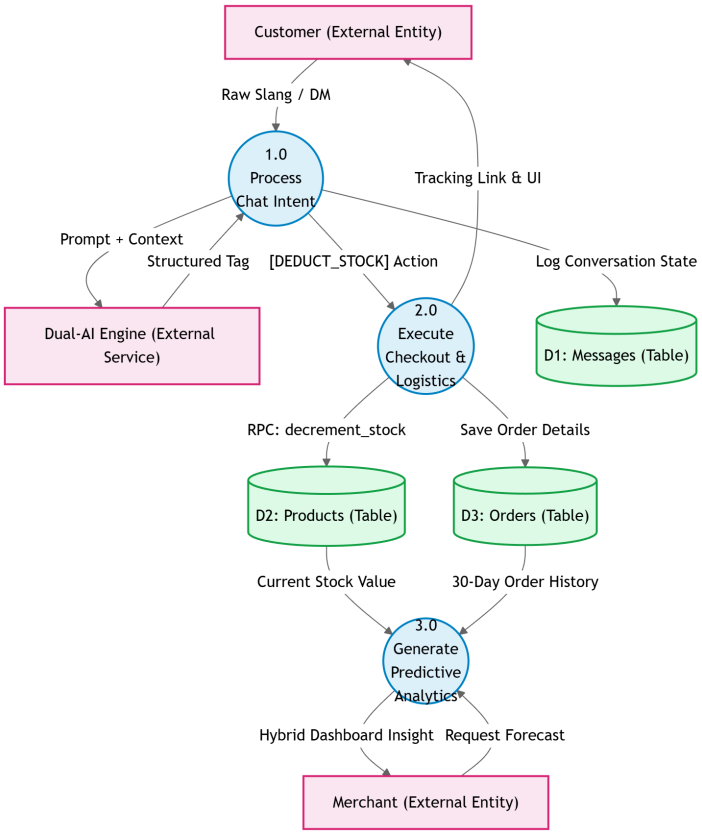

Figure 2.4: SellerMate Data Flow Diagram illustrating system processes and data stores (Zero Black Holes or Miracles)

© UMHackathon 2026

2.3.2. Normalized Database Schema (3NF)

The Entity Relationship Diagram (ERD) below visualizes the logical data model. A key highlight of this architecture is the implementation of the ORDER_ITEMS entity. This acts as a junction table designed specifically to resolve the inherently Many-to-Many (M:N) relationship between ORDERS and PRODUCTS. By isolating the quantity and unit_price_at_purchase into this junction table, the system prevents transitive dependencies and guarantees that historical order receipts remain accurate even if a merchant updates the current price of a product in the future.

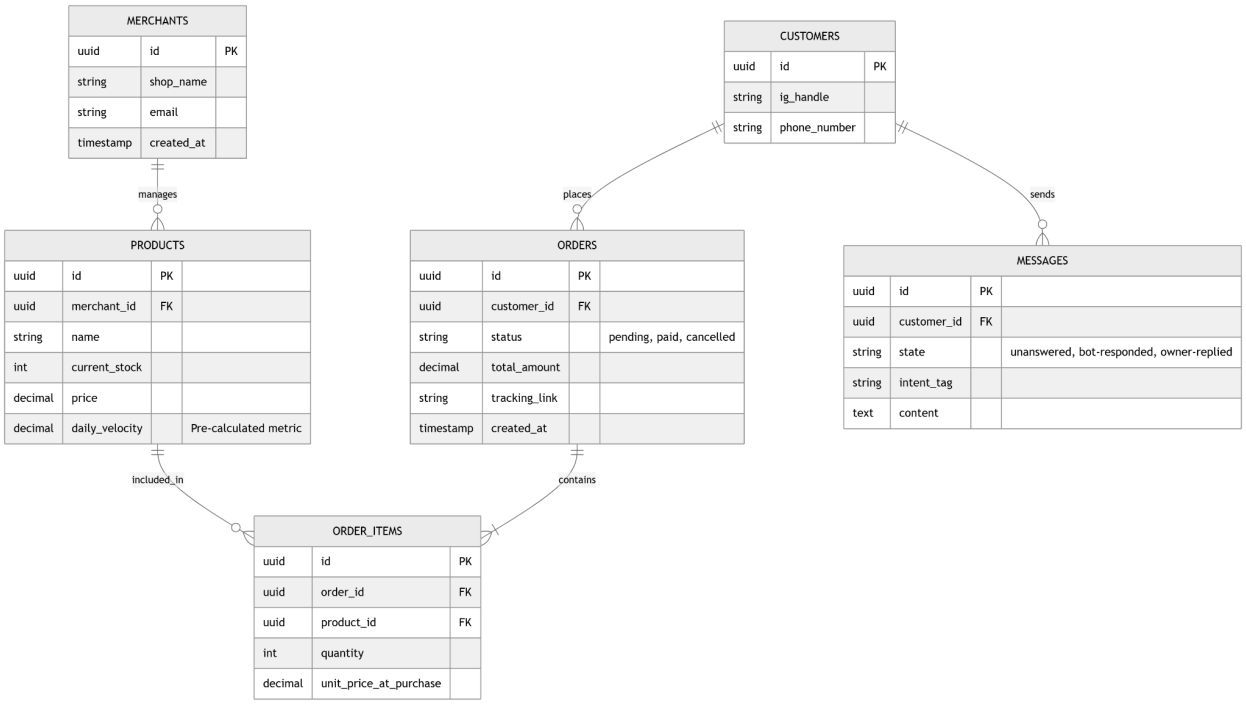

Figure 2.5: Logical Entity Relationship Diagram (ERD) designed in strict Third Normal Form (3NF)

© UMHackathon 2026

3. Functional Requirements & Scope

This section highlights the core boundaries of the SellerMate project. This enables the development team to focus strictly on a high-impact Minimum Viable Product (MVP) to showcase the technical feasibility and architectural superiority of the platform during the evaluation.

3.1. Minimum Viable Product:

3.1.1. Core Features for UMHackathon 2026 Demo

The team is committed to building and demonstrating the following 5 core functional pillars by the end of the UMHackathon 2026:

Table 3.1: Core MVP Features for UMHackathon 2026 Demonstration

#	Features	Description
1	**Priority-based AI Conversational Checkout**	Utilizes a high-availability failover architecture (Z.ai Primary, Groq Backup) to process complex Malaysian slang and instantly generate structured checkout links via the simulated IG UI.
2	**3-Strike Bot Cut-off & HITL**	Monitors the entire AI failover chain, enforcing a strict 3-attempt limit on total failures before autonomously locking the bot session and alerting the merchant for a manual override.
3	**Atomic Inventory & Rollback**	Ensures data integrity via atomic database transactions, performing instant stock deductions on checkout and automated inventory rollbacks upon UI-triggered order cancellations.
4	**Pure AI-Driven Predictive Dashboard**	A B2B dashboard where the AI engine directly analyzes raw sales data to predict stock depletion, detect anomalies, and generate operational briefings without relying on local mathematical fallbacks.
5	**Intelligent Logistics & Tracking**	Uses AI browser tools to fetch and compare live courier rates for optimal margins, automatically generating localized tracking links for seamless fulfillment.

© UMHackathon 2026

3.2. Non-Functional Requirements (NFRs)

This section highlights the critical system qualities that dictate how well the SellerMate platform performs. These requirements are engineered to ensure enterprise-grade reliability, cost-effective AI scaling, and an uninterrupted social commerce experience.

Table 3.2: Non-Functional Requirements (NFRs) and Implementation Strategies

Quality	Requirements	Implementation
Scalability	The system must handle sudden, massive traffic spikes, for example resulting from a viral Instagram post without dropping incoming chat messages or checkout requests.	Deployed on **Vercel's Edge Network** using Next.js Serverless API routes for automatic, instant scaling. **Supabase connection pooling** is utilized to manage high concurrent database hits.
Reliability	The system must guarantee 100% data consistency during checkout. Concurrent purchases must never result in negative inventory (overselling).	Enforces strict **ACID compliance** via PostgreSQL. Utilizes atomic **Remote Procedure Calls (RPCs)** such as decrement_stock to lock and update database rows safely.
Maintainability	The codebase must be highly modular, allowing the team to swap AI models or update prompt logic without breaking the frontend UI.	Adopted a **modular serverless architecture**. AI orchestration logic, external API integrations (Logistics), and database controllers are decoupled into isolated directories.
Token Latency	The Dual-AI conversational bot must parse local slang and respond within a strict timeframe to prevent users from abandoning the chat.	Implemented a 40-second **strict circuit breaker**. Utilizes **Groq's LPU** for ultra-fast token generation, ensuring millisecond-level Natural Language Understanding (NLU).
Cost Efficiency	AI API token consumption must be strictly minimized during dashboard predictions to ensure the B2B business model remains highly profitable.	Employs **Hybrid Reasoning**. The backend pre-calculates mathematical metrics and injects highly **compressed string payloads** into Z.ai, drastically reducing input token costs per query.

© UMHackathon 2026

3.3. Out of Scope / Future Enhancements

The SellerMate development team recognizes that a "Grand Vision" platform requires extensive post-hackathon iteration. The following high-level features are strategically excluded from the UMHackathon 2026 MVP to maintain a focused, high-impact demonstration of the Dual-AI architecture:

a. **Autonomous Rider Dispatching System:** The current scope focuses on the merchant-side operational intelligence. Complex GPS-based real-time rider tracking and dynamic dispatching algorithms are reserved for the "Scale-Up" phase.

b. **Complex Multi-Vendor Loyalty Engine:** While the system currently handles inventory and orders seamlessly, sophisticated promotional engines, loyalty point tiers, and dynamic discount code generation are out of scope for the MVP.

c. **Multimodal AI Integration (Vision & Audio):** The current MVP is text-centric. Future iterations will integrate vision-language models like GPT-4o or similar vision encoders to enable the AI to process customer-submitted images such as screenshots of cart items, damaged product photos and voice notes (Speech-to-Text), creating a truly intuitive social commerce experience.

d. **Automated Refund & Dispute Resolution:** While the current system handles order cancellations, automated AI-driven dispute mediation such as assessing photo evidence of damaged goods is deferred to ensure focus on the core checkout flow.

e. **Live-Stream Automation (Real-time Intent Detection):** The current MVP focuses on text-based DM processing. Future iterations will introduce Real-time Live-Stream Automation, where the AI monitors live video commentary. It will autonomously detect intent patterns, for example, users typing "1" or "PM" and instantly trigger personalized private messages to those customers with specific product details, bridging the gap between live engagement and automated conversion.

© UMHackathon 2026

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. Grayscale Rollout & A/B Testing:

We utilize a phased deployment approach to minimize production impact. New updates move through these stages:

- **Grayscale Rollout:** New features deploy to 5% of merchants first to monitor latency and AI hallucinations. Upon passing sanity checks, traffic is scaled to 25%, 50%, and finally 100%.
- **Failover & Latency Testing:** Once a feature reaches 100% rollout, we actively test the reliability of the priority-based failover. We simulate Z.ai timeouts to evaluate Groq's backup response time and intent-parsing accuracy, ensuring the 40-second circuit breaker functions correctly under stress.

4.1.2. Emergency Rollback (ER) & Golden Release:

- **Automated Rollback:** The CI/CD pipeline tracks telemetry for 5xx errors or bot failures. Any significant spike triggers an immediate, automated rollback.
- **Golden Version:** We maintain a "Golden Version" of the most recent stable release. The system overwrites failed deployments with this version to restore service within seconds.

4.1.3. Priority Matrix:

Table 4.1: Operational Priority Matrix and Mitigation Strategies

Priority	Trigger Condition	Required Action
P1: Critical	Checkout link generation failure exceeds 1% OR total system database outage.	Trigger instant Emergency Rollback; alert development team; disable AI checkout generation to prevent customer churn.
P2: High	Failover chain fails completely (3-strikes hit) for 10% of sessions within a 5-minute window.	Lock bot session; force handoff to Human-in-the-Loop; flag logs for prompt engineering audit.
P3: Medium	Primary AI (Z.ai) latency exceeds 40 seconds or returns 5xx error.	**Automatic Failover:** Switch to backup AI (Groq). If analytics dashboard queries fail, gracefully degrade UI to "Prediction Unavailable" state.

© UMHackathon 2026

P4: Low	Cosmetic UI bugs or minor text inaccuracies in chat response.	Log the issue in the tracker; schedule for resolution in the upcoming hotfix release cycle.

4.2. Monitoring:

The system health is tracked through real-time telemetry focusing on API responsiveness and AI accuracy.

- **Infrastructure Health:** Vercel Edge Logs and Supabase Monitoring detect HTTP 5xx server errors and database connection latency in real time.
- **AI Performance Trigger:** The strict 40-second circuit breaker serves as the primary health trigger for AI inference.
- **Safe Failure Management:** Total inference failure across both AI providers beyond the 40-second window forces an immediate UI state change such as displaying "Service Unavailable", halting automated processes without crashing the backend.
- **Dialogue Quality Monitor:** System health regarding conversation quality is tracked via the 3-strike logic. A session failure count reaching three triggers an immediate system alert on the merchant dashboard for manual intervention.

4.3. Assumptions:

The development and deployment of SellerMate are based on the following operational and environmental conditions:

- **Merchant Connectivity:** Merchants utilize modern hardware and stable internet connections to access the web-based B2B dashboard.
- **Customer Environment:** Customers possess active social media accounts and sufficient mobile data to interact with the chat interface.
- **Infrastructure Availability:** External services including Groq LPU inference and Supabase Cloud remain operational and accessible.
- **Data Accuracy:** All product inventory and pricing information provided by the merchant is accurate and maintained within the database.
- **Logistics Portals:** Regional shipping providers maintain public-facing web portals for automated rate fetching and tracking.

© UMHackathon 2026

4.4. External Dependencies:

The functionality of SellerMate relies on the following external services and infrastructure components:

Table 4.2: External Dependencies, Risk Assessment, and Mitigation

Tools	Purpose	Risks
Priority-based AI Engine (Z.ai & Groq)	Orchestrates natural language parsing, intent recognition, and browser-based logistics rate comparison.	**High Risk:** Complete provider downtime paralyzes core features. **Mitigation:** Automated failover from Z.ai to Groq. If both fail, trigger HITL 3-strike cut-off and safe UI degradation.
Supabase Cloud	Provides ACID-compliant PostgreSQL storage for managing inventory, orders, and message history.	**Medium Risk:** Database bottlenecks during viral traffic surges. **Mitigation:** Utilize strictly optimized atomic RPC calls for instant transaction processing and locking.
Vercel Edge	Manages Next.js serverless execution and global delivery of the platform.	**Medium Risk:** Execution timeouts during complex AI orchestration tasks. **Mitigation:** Enforce the 40-second circuit breaker and utilize asynchronous background logging.

© UMHackathon 2026

5. Project Management & Team Contributions

5.1. Project Timeline

The project was developed using an Agile methodology, where tasks such as requirement analysis (PRD), system design (SAD), development, testing, and presentation preparation were carried out iteratively and in parallel. This approach allowed the team to continuously refine system features, improve AI outputs, and adapt to time constraints while ensuring steady progress throughout the 10-day sprint.

Table 5.1: Project Timeline (10-Day Sprint Plan)

Day	Phase	Activities	Justification
Day 1	Ideation & Domain Discussion	- Discussed project direction (social commerce vs e-commerce) - Identified real-world problems (manual workload, slow replies, stock issues)	Ensures the team solves a real and relevant problem before building
Day 2	Requirement Planning (PRD)	- Created Product Requirement Document (PRD) - Defined use cases and user stories - Identified system scope and core features	Provides a clear functional foundation and avoids scope creep
Day 3	System Design (SAD)	- Developed System Analysis & Design (SAD) - Designed system architecture (chat → AI → database → workflow) - Planned database schema (orders, inventory, products)	Establishes a scalable and structured system design before development
Day 4-6	Core Development (Backend + AI	- Evaluated AI APIs (Z.AI, Groq) - Designed prompts for	- Critical for ensuring AI accuracy, reliability, and fallback strategy

© UMHackathon 2026

	Integration)	chat handling and analytics - Implemented chat processing logic - Integrated AI APIs - Built order extraction and inventory validation	- Focus on delivering Minimum Viable Product (MVP)
Day 7	Feature Enhancement	- Added AI analytics (daily briefing, insights, action plan) - Implemented fallback AI logic	Enhances system with value-added intelligent features
Day 8	Testing & Iteration	- Conducted unit & integration testing - Fixed bugs and improved AI prompts	Agile iteration improves system stability and quality
Day 9	QA & Optimization	- Performed performance testing - Refined workflows and fallback handling - Improved response accuracy	Ensures performance, reliability, and consistency
Day 10	Finalization & Documentation	- Prepared QA documentation - Created pitch deck - Filmed pitching video - Final system validation	Ensures complete deliverables for evaluation

© UMHackathon 2026

5.2. Team Members Role

Table 5.2: Team Members & Roles

Member	Role	Responsibilities
Heng Hon Lim	Backend Lead	Designed database schema, implemented order & inventory management, handled API integration
Chin Chun Wei	AI Engineer	Designed prompts, integrated Z.AI & Groq APIs, implemented fallback mechanism and AI workflows
Tam Hui Shan	QA & Testing Lead	Developed test cases, performed system testing, validated AI outputs and workflow reliability
Varunesh Rau A/L Bala Thandayuthabani	System Designer / Frontend	Designed system flow, user interaction logic, and assisted in integration and presentation

5.3. Recommendations

5.3.1. Scalability

- Implement message queue system (e.g., Kafka / RabbitMQ) to handle high volumes of chat requests asynchronously
- Use microservices architecture: Separate AI processing, order management, and analytics services

5.3.2. Performance

- Introduce caching (e.g., Redis) for frequently accessed product data and inventory status
- Optimize AI calls by reducing unnecessary prompt size and reusing context where possible

5.3.3. Reliability

- Add monitoring & alert system to detect API failures or abnormal system behavior
- Implement data validation layer to prevent incorrect AI outputs from affecting system decisions

5.3.4. Future Improvements

- Support integration with more platforms for real deployment
- Provide live chat analytics
- Multimodal AI Integration (Vision & Audio)

© UMHackathon 2026